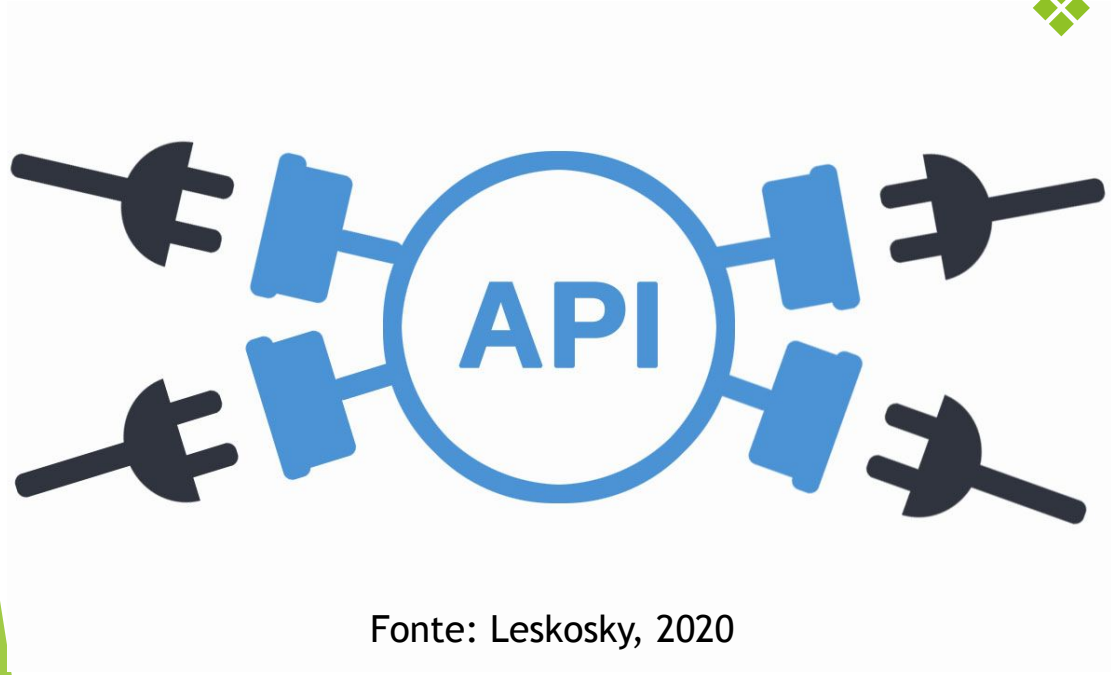
Desenvolvimento Web

Profª Sabrina B. Moreira

Conceitos avançados

Aula 4.1

API Application Programming Interface



Fonte: Leskosky, 2020

- Uma API (Interface de Programação de Aplicações) é um conjunto de regras que permite que diferentes sistemas ou programas se comuniquem entre si.

API Application Programming Interface

Pode ser entendida como:

- Um conjunto de funções prontas que permitem acessar um **serviço** ou solicitar **dados de outro sistema**, facilitando o desenvolvimento de aplicações.

Uma API normalmente disponibiliza várias operações, como:

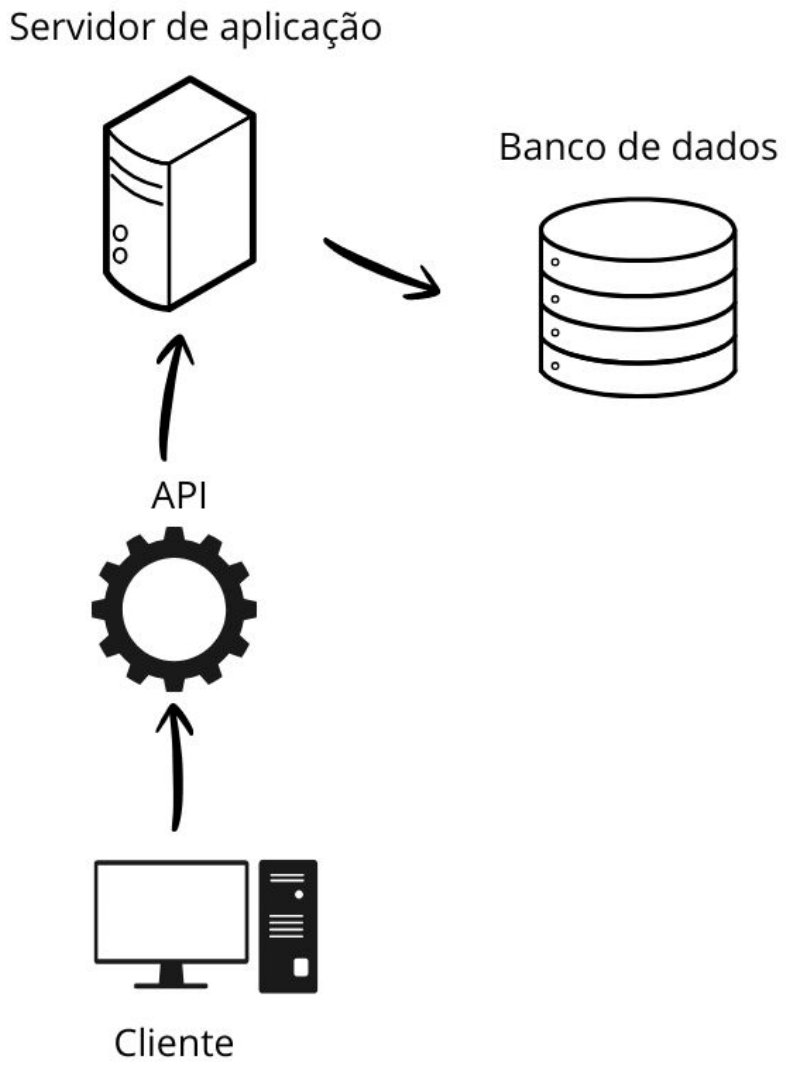
- consultar dados
- enviar informações
- atualizar registros
- excluir dados

Essas operações ficam disponíveis através de endereços específicos chamados endpoints.

API Application Programming Interface

- O <u>cliente</u> é quem faz a **requisição** de dados ou ação à API.

- O <u>backend</u> é a parte do sistema que **processa** as solicitações, **executa regras de negócio** e **acessa os dados no banco** de dados.



Fonte: autora, 2026

API Application Programming Interface

No backend normalmente ficam:

- banco de dados
- lógica de negócio
- autenticação
- processamento de dados

Quando a API recebe um pedido, ela **interage** com o backend para buscar ou alterar informações.

Como Cliente, API e Backend interagem

- 1- Cliente envia a **requisição** à API
- 2- API **processa o pedido**, verifica se é válido, acessa o backend e o banco de dados.
- 3- O backend **processa os dados** e retorna a API, que envia a resposta ao cliente.

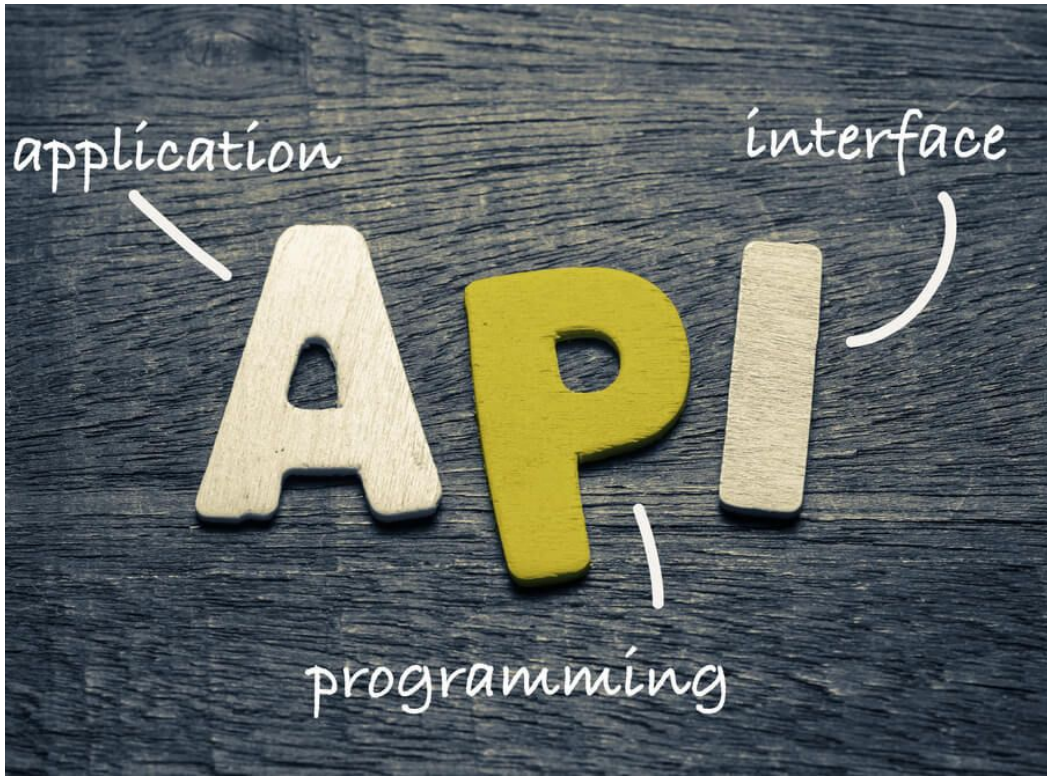
Resumo: backend

Chamamos de backend, tudo que **roda no servidor** e o cliente **não vê**, como:

- Regras de negócio
- Acesso ao banco de dados
- Autenticação
- Processamento de dados
- Integrações com outros sistemas
- APIs

Ou seja, a API faz parte do backend, mas backend não é apenas API.

Resumo: API

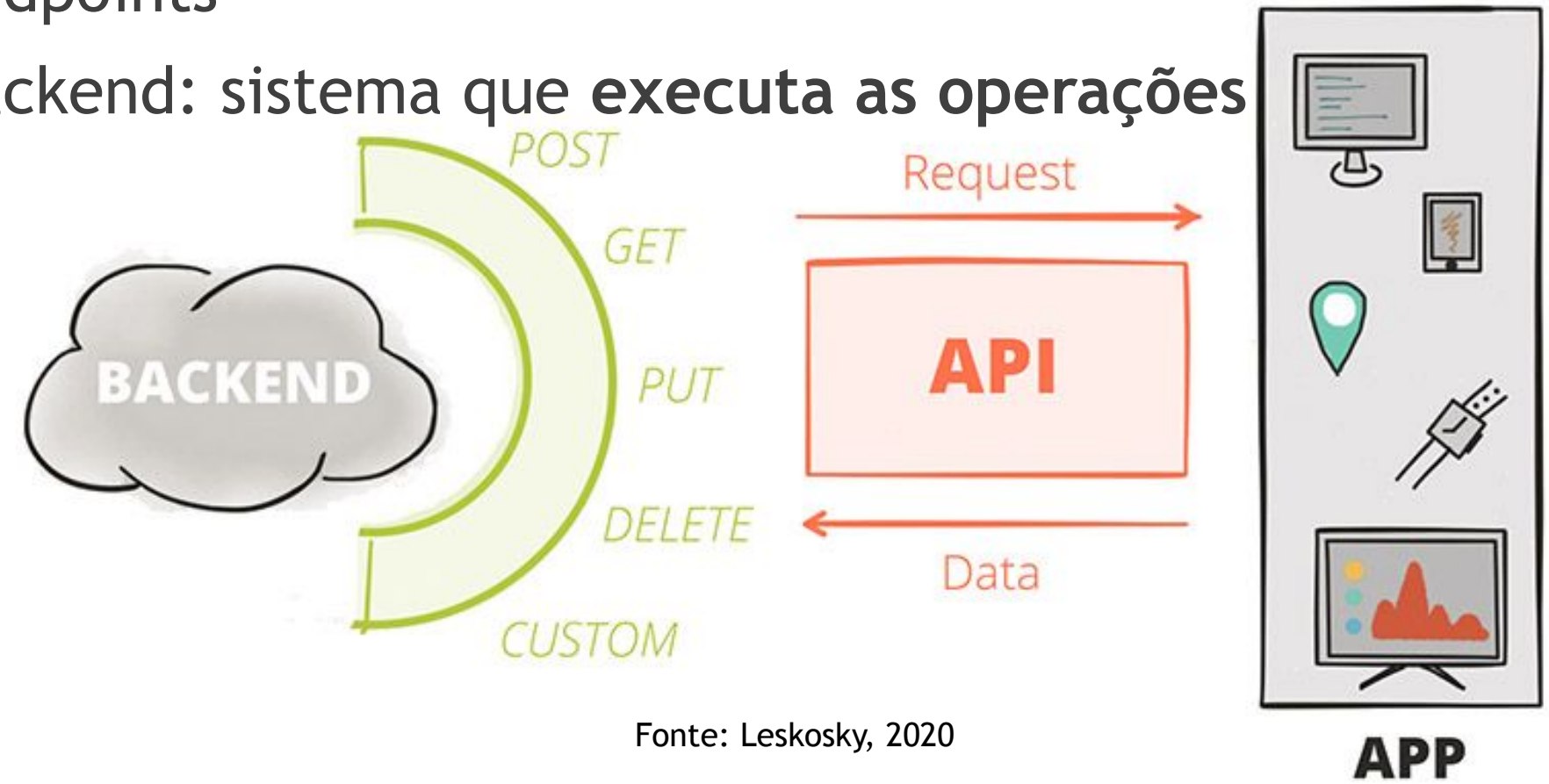


Fonte: neilpatel

- A API é a **porta de entrada** do backend.
- Ela define **como** o cliente pode conversar com o backend.

Diferença de API e Backend

- API: é a **interface de comunicação** que expõe os endpoints
- Backend: sistema que **executa as operações**



Fonte: Leskosky, 2020

Conceito de endpoint

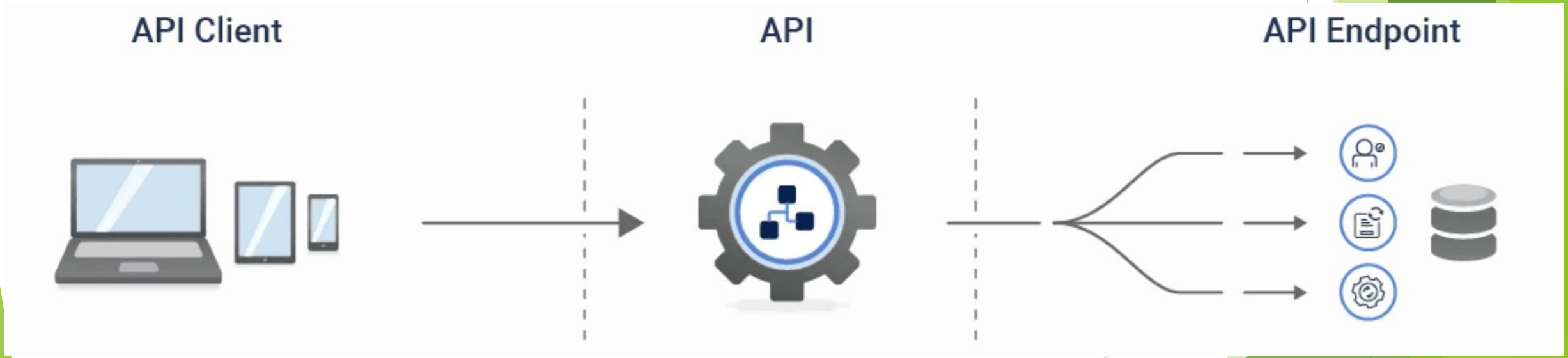
Um endpoint é um endereço específico dentro de uma API que permite **acessar um recurso** ou **executar uma operação**.

Em APIs web, esse endereço geralmente é uma URL associada a um método HTTP, como:

- GET
- POST
- PUT
- DELETE

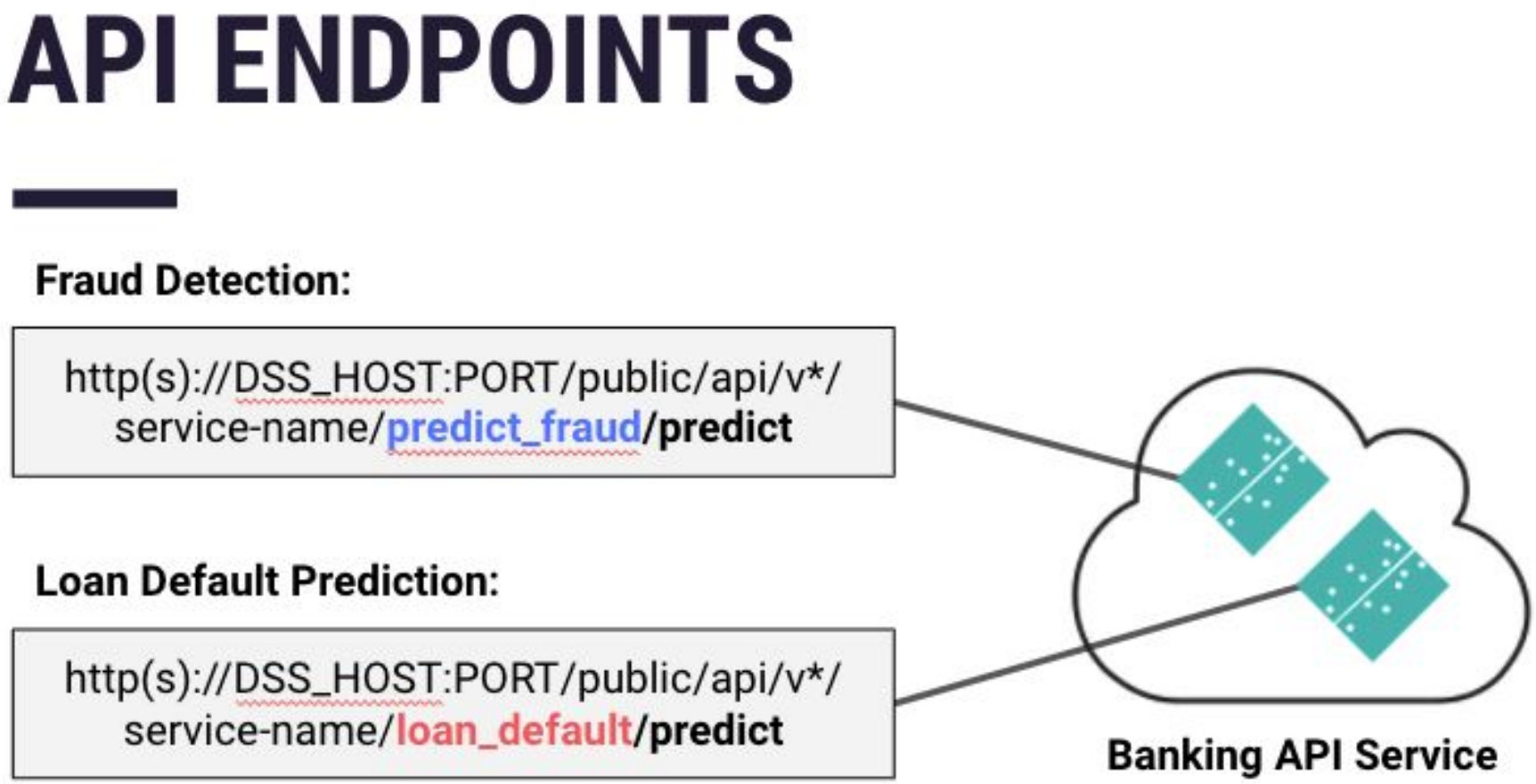
Conceito de endpoint

- Cada endpoint representa um **ponto de acesso** para uma <u>funcionalidade</u> da API.



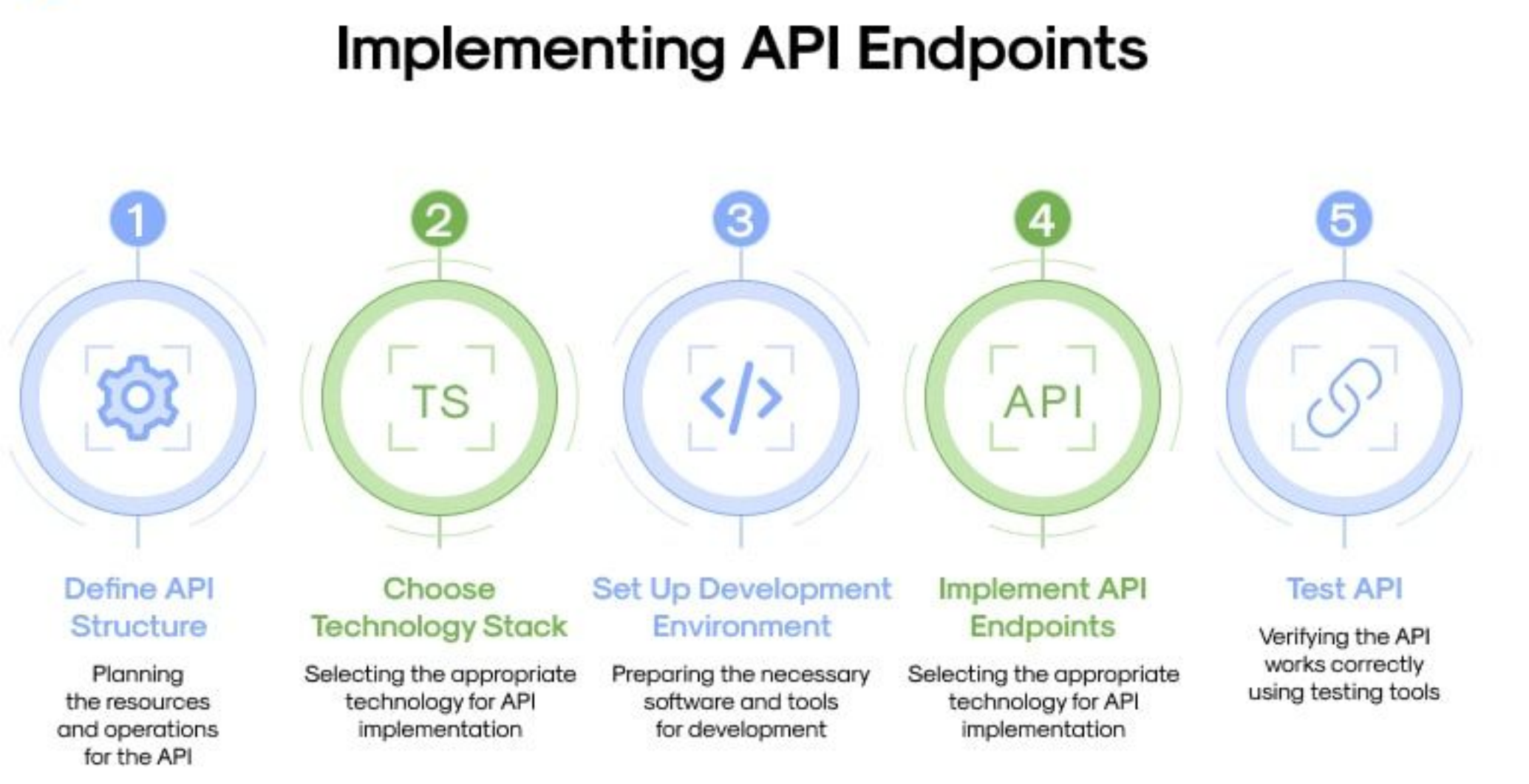
Fonte: Srivastava, 2024

Diferença de API e Endpoint



Fonte: gotpi

Desenvolvimento de endpoints



Fonte: Ilyin, 2025

Exemplo prático: API da Nasa

Software utilizado para as requisições: Insomnia

Link para as APIs Públicas da NASA: NASA OPEN API

APIs:

- APOD: foto astronômica do dia: link
- Asteroids - NeoWs > Neo - Feed: Listagem de asteróides de acordo com sua proximidade da terra: link
- InSight: Mars Weather Service API: link

E onde entra o front-end nisso tudo?

O frontend é a parte do sistema com a qual o usuário interage diretamente.

Ele roda no:

- Navegador
- Aplicativo mobile
- Interface gráfica

Frontend

Suas principais responsabilidades são:

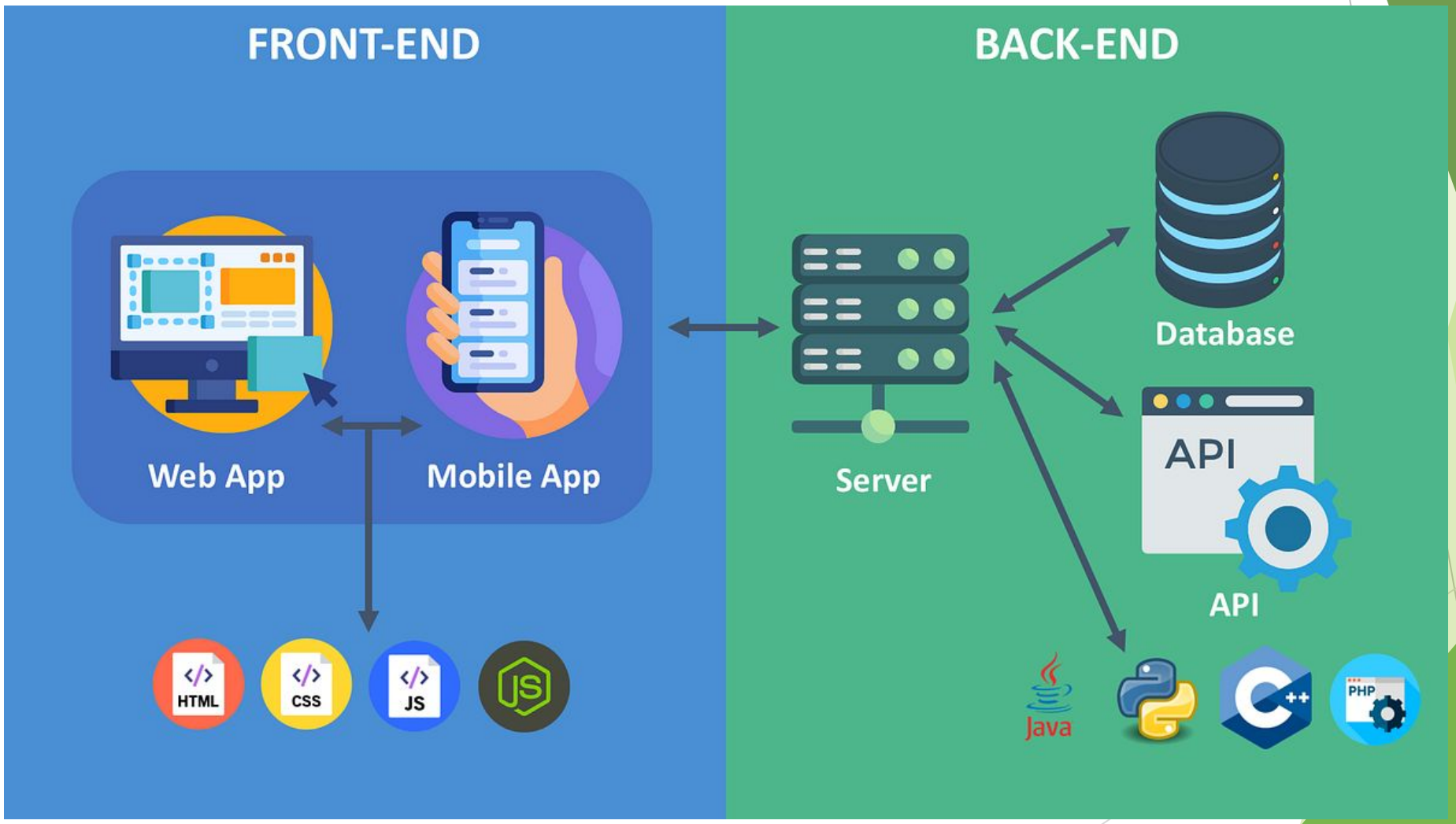
- Exibir telas e informações
- Receber ações do usuário (cliques, formulários, etc.)
- Enviar requisições para o servidor

Tecnologias comuns de frontend incluem:

- Angular
- React
- Vue.js
- HTML, CSS e JavaScript

Fonte: wikipedia

Desenvolvimento Front-end x Back-end



Fonte: freepik

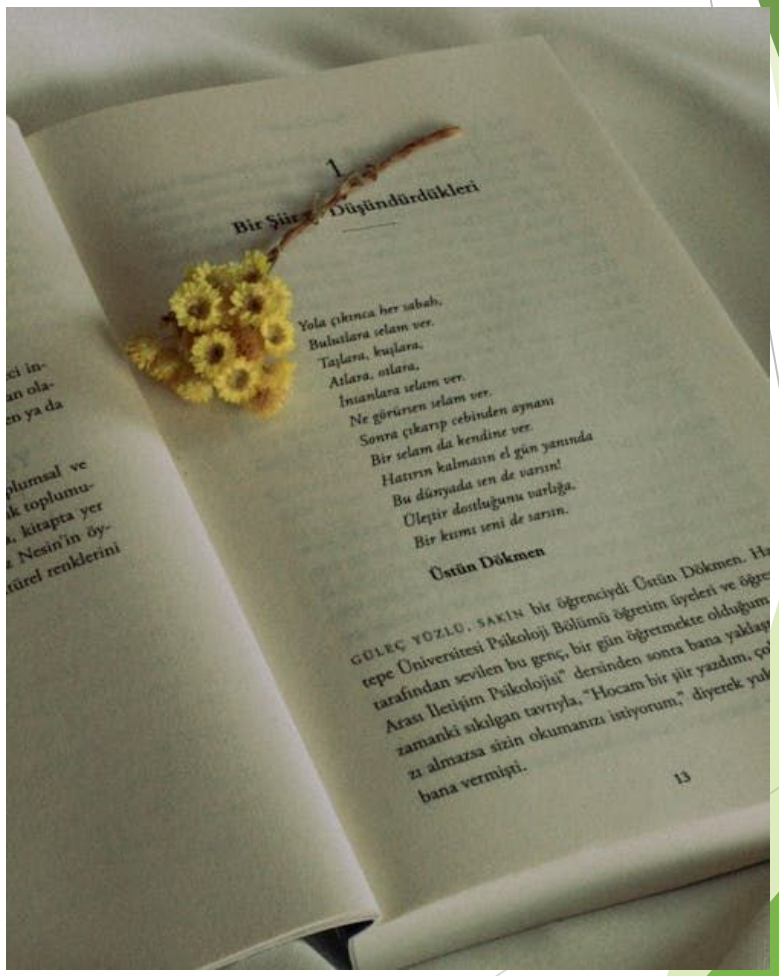
HTML e CSS

Aula 4.2

Hipertexto: navegação linear x não linear

- Antes da criação do hipertexto, tínhamos somente a **navegação linear** entre os documentos.

- Como por exemplo, na leitura de um livro ou jornal folheamos página por página, de forma linear e sequencial por meio de documentos físicos.



Fonte: Korkmaz

Hipertexto: hipertexto

- E em 1989 Tim Berners-Lee teve uma ideia simples e poderosa:

Criar documentos científicos que pudessem se referenciar mutuamente por meio de links.

Isso mudou completamente a forma como o conhecimento é consumido nos dias atuais.

Hipertexto: o que é

- É um **modelo de organização de informações baseado em conexões**, ou links, entre documentos.
- E o HTML surgiu para estruturar documentos que pudessem ser interligados por hiperlinks, sendo uma ferramenta para.
- A tag <a> do html, cria um hiperlink que permite a navegação para outras páginas ou documentos.

```
<a href="https://exemplo.com">Clique aqui</a>
```

Clique aqui

Fonte: autora, 2026

HTML: o que é

- O HTML (HyperText Markup) é uma <u>linguagem de marcação</u> que cria a **base estrutural** para uma página que é exibida em um navegador.

- É formada por um **conjunto de tags** que permite exibir textos e hipermídia, como links, imagens, tabelas, vídeos etc…

- Entretanto, **apresenta poucos recursos** de formatação visual, e por isso *é utilizado em conjunto com o CSS*.

HTML: função

O que o HTML faz:

- Organiza conteúdo
- Define estrutura
- Indica significado semântico
- Diz ao navegador o que é cada coisa

E o que não faz:

- Estruturas de controle
- Algoritmos
- Execução de instruções

Para isso utilizamos o JavaScript.

HTML: histórico

- **1990**: criado por Tim Berners-Lee na Suíça, junto com o HTTP e primeiros navegadores no projeto de construção da World Wide Web (WWW).

Fonte: Veja, 2011

HTML: histórico

HTML

2.0

Fonte: autora, 2026

- **1991-1993**: HTML 1, primeira versão com conjunto simples de tags pensado para atender projetos estáticos de hipertexto.
- **1995**: surgiu o HTML 2, trouxe maior padronização de recursos e os formulários.

HTML: histórico

- **1997**: O W3C (World Wide Web Consortium, organização internacional de padronização da web, lançou a versão 3.2 com suporte para tabelas, recurso amplamente usado na época.

Fonte: w3.org

HTML: histórico

HTML

4.01

- **1999**: suporte ampliado ao CSS, estrutura mais clara e com preocupações de acessibilidade.
- **2000**: reformulação do HTML 4 para seguir as regras rígidas do XML (XHTML), atribuido erros de sintaxe e quebra de arquivo a escritas incorretas, como tags que ficaram abertas, sem aspas ou escritas em maiúsculo. Mas o padrão foi considerado muito rigoroso e intolerante a erros.

Fonte: autora, 2026

HTML: histórico

- **2004-2014:** o primeiro rascunho do HTML 5 saiu em 2008, pensando em voltar atrás no padrão XHTML a lançou uma nova versão criando elementos semânticos de estruturação de conteúdo, multimídia nativo e apis avançadas para geolocalização e canvas 2d.

HTML

5

Fonte: autora, 2026

HTML: Padrão contínuo (living standard)

Padrão vivo

- Desde a unificação dos esforços do WHATWG e W3C em torno de HTML5, o padrão passou a ser mantido como "HTML Living Standard", ou seja, um conjunto de especificações que evoluem continuamente sem versões numeradas como HTML6.
- O HTML recebe atualizações graduais e suporte a novos recursos sem que a linguagem precise redefinir “versões”.

HTML: exemplo

- Código

```html
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Minha Página</title>
</head>
<body>
  <h1>Meu Título</h1>
</body>
</html>
```

- Exibição

Meu Título

Fonte: autora, 2026

HTML: entendendo as tags

- <!**DOCTYPE** html>: Define que é HTML5
- <html **lang**="pt-BR">: Define o idioma da página
- <meta **charset**="UTF-8">: Permite acentuação
- <**title**>: Título que aparece na aba do navegador
- <h1>: Título de maior peso, exibido na página

HTML: um pouco sobre o UTF…

- No nosso exemplo, a tag UTF indica o navegador a Interpretar os caracteres deste arquivo usando o padrão UTF-8.

- Caso não seja indicado o UTF correto, podemos ter esse tipo de erro em nossos arquivos:

Você escreve:

- **Programação é incrível**

E na tela aparece:

- **ProgramaÃ§Ã£o Ã© incrÃvel**

HTML: as principais tags

As principais tags HTML são:

- **Head**: contém as configurações da página, ficando invisível ao usuário (exceto o que está na tag <title>)

- **Body**: envolve todo o conteúdo que o usuário vê, como textos, mídias, formulários, menus…

CSS: o que é

- CSS (Cascading Style Sheets) é uma **linguagem de estilo** usada para definir a aparência e o layout de páginas web.

Fonte: freepics

CSS: o que é

- Ele controla <u>como</u> os elementos de uma página HTML **serão exibidos**, como cores, fontes, espaçamento, tamanhos e posicionamentos.

- Enquanto o **HTML** <u>define a estrutura</u> da página (títulos, parágrafos, imagens, links), o **CSS** define <u>como esses elementos serão apresentados</u> visualmente ao usuário.

CSS: função

- O CSS, ou folha de estilo, permite enriquecer o seu arquivo HTML com **diferentes formações** para texto, bordas, fundos e animações.

```
body {
  background-color: lightgreen;
  background-image: url('some-example-image.jpg');
  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;
}
```

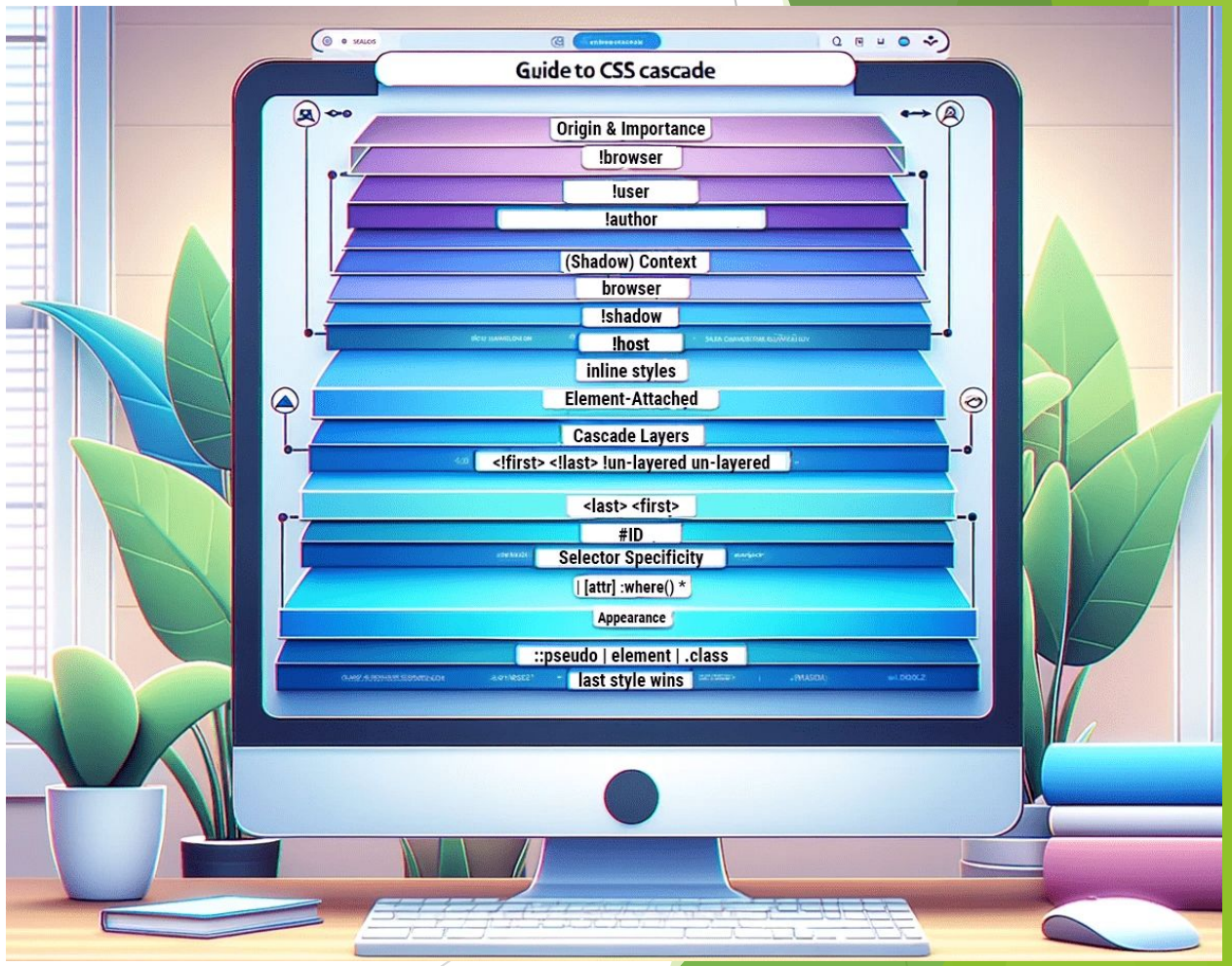
Fonte: JetThoughts, 2022

CSS: Característica “Cascading” (Cascata)

O termo cascading significa que as regras de estilo podem se **sobrepor** ou **herdar** propriedades.

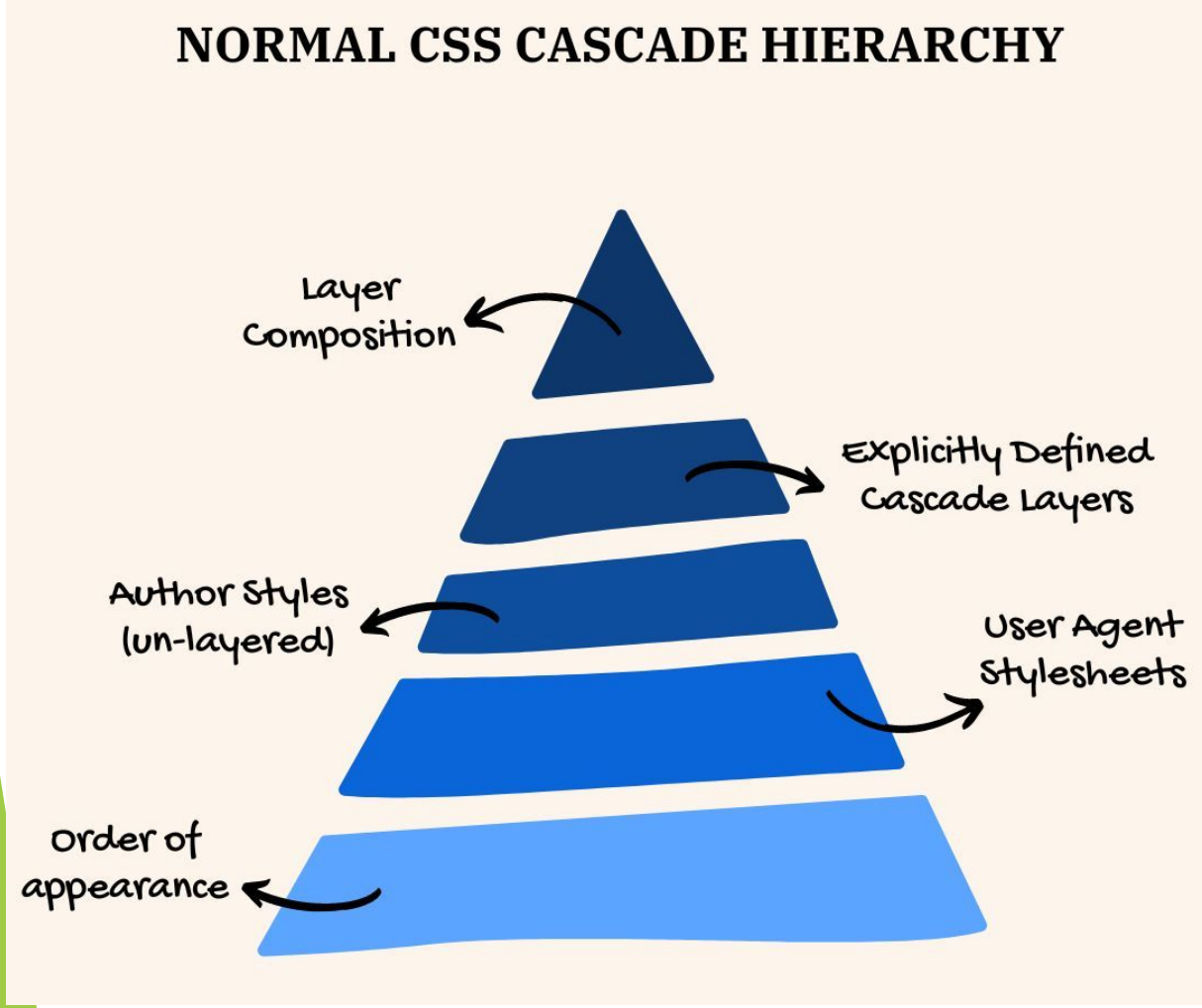
Isso permite que:

- estilos gerais sejam definidos para toda a página
- estilos específicos modifiquem apenas alguns elementos



Fonte: Joy, 2024

CSS: Característica “Cascading” (Cascata)



Fonte: Joy, 2024

Quando vários estilos afetam o mesmo elemento, o CSS precisa decidir qual regra tem prioridade e para isso várias regras são verificadas.

CSS: Característica “Cascading” (Cascata)

Que é definida por:

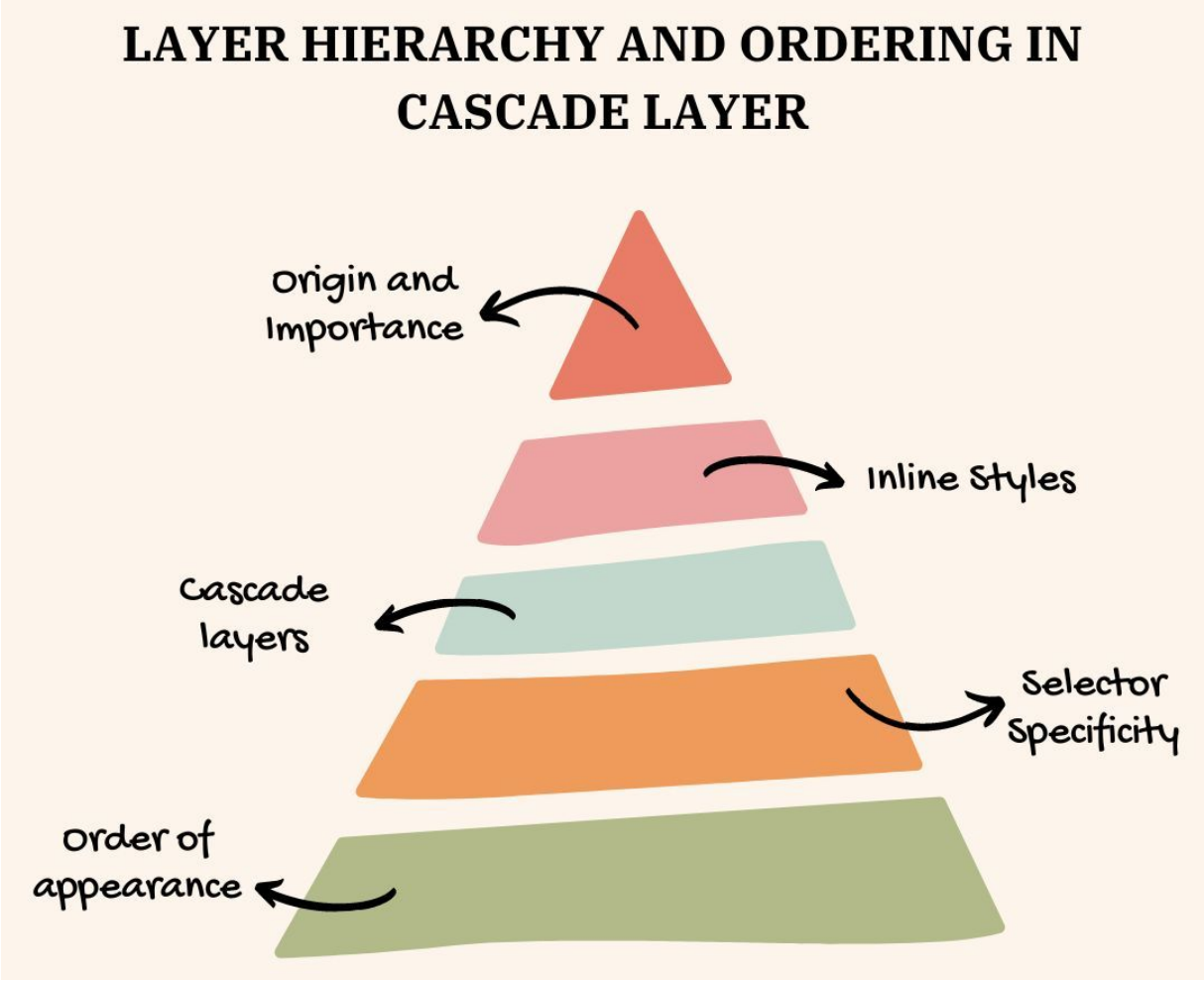
- Origem do estilo (User Agent Styles ou Author Styles)
- Camadas (cascade layers, qual prioridade da cama)
- Especificidade do seletor (qual seletor utilizado)
- Ordem em que o estilo aparece no código (mais a baixo vence)
- Uso de !important (sobrescreve quase qualquer regra)

```
@layer base {
  p {
    color: black;
  }
}

@layer components {
  p {
    color: blue;
  }
}
```

Fonte: autora, 2026

CSS: Especificidade do seletor



Fonte: Joy, 2024

Hierarquia simplificada de especificidade:

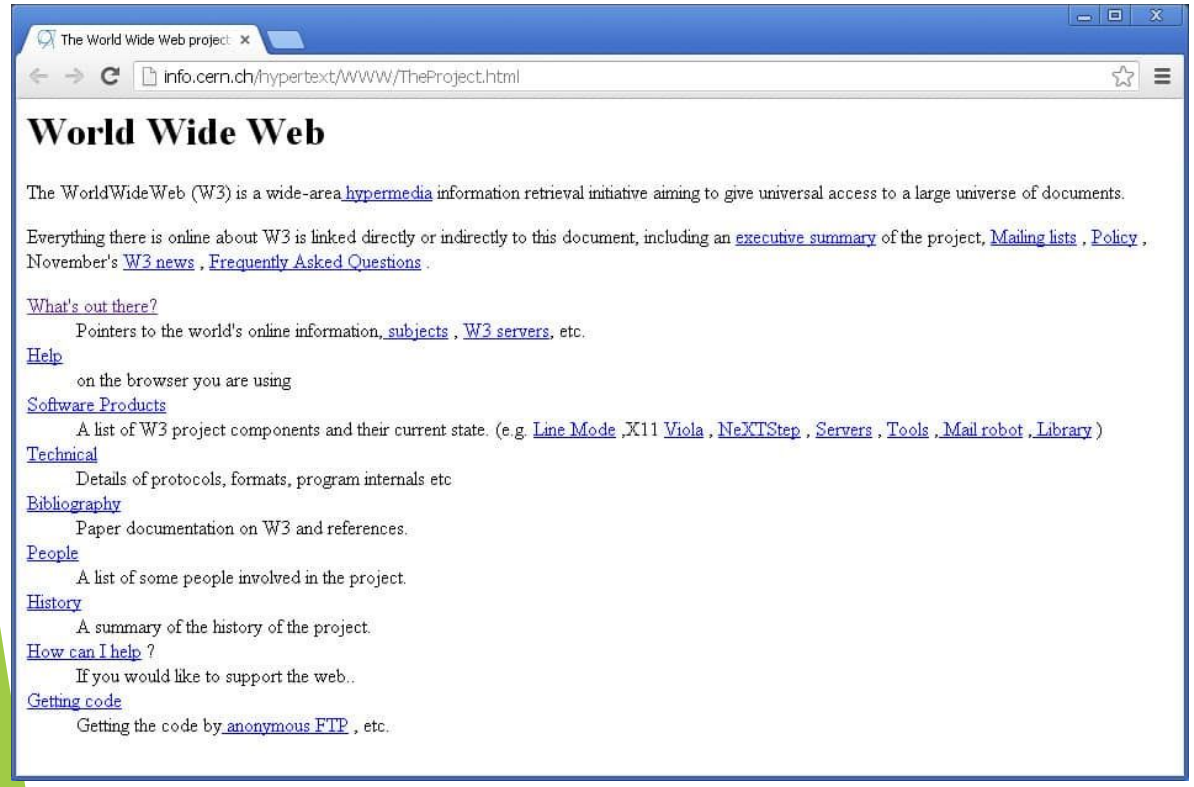
- seletor de elemento
- classe
- id
- estilo inline

CSS: Especificidade do seletor

Menor peso ao maior:

- Estilos padrão do navegador
- Estilos do autor em camadas inferiores
- Estilos do autor em camadas superiores
- Estilos sem camada (unlayered styles)
- Estilos inline
- !important

CSS: histórico



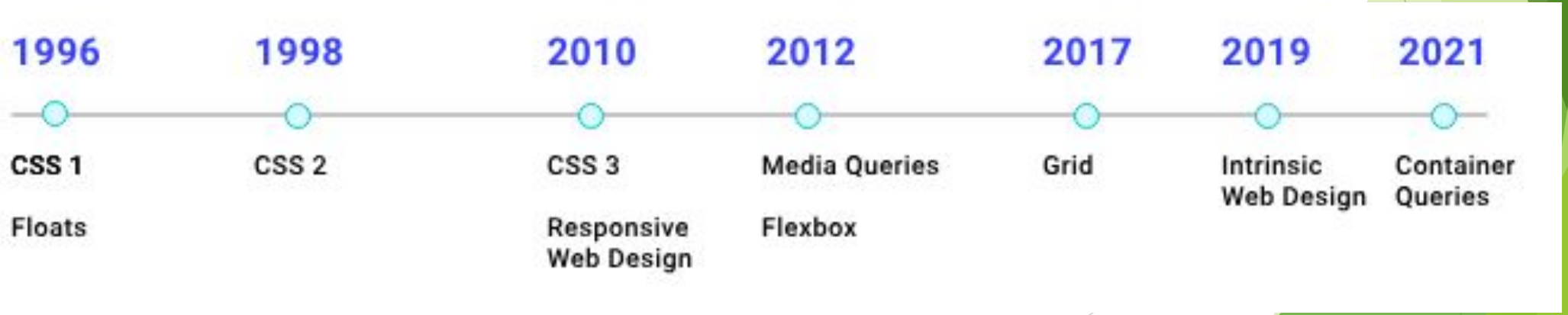
Fonte: Bell, 2021

O CSS foi criado para resolver um problema importante no início da web:

As páginas HTML misturavam conteúdo e formatação, o que tornava o desenvolvimento difícil.

CSS: histórico

- Foi proposto em 1994 pelo cientista da computação Håkon Wium Lie, enquanto trabalhava no CERN com o criador da web Tim Berners-Lee.
- A ideia era permitir que o design das páginas fosse controlado separadamente do conteúdo.



Fonte: Bell, 2021

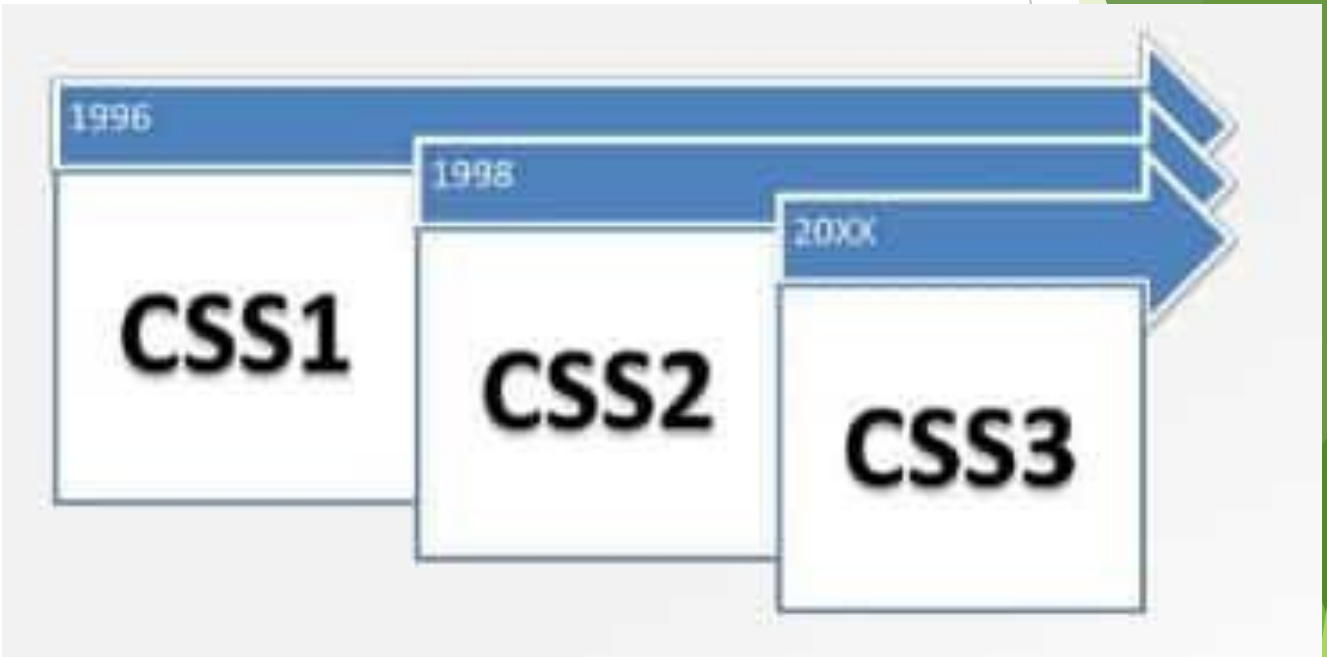
CSS: histórico

CSS1 (1996)

- Primeira versão oficial.

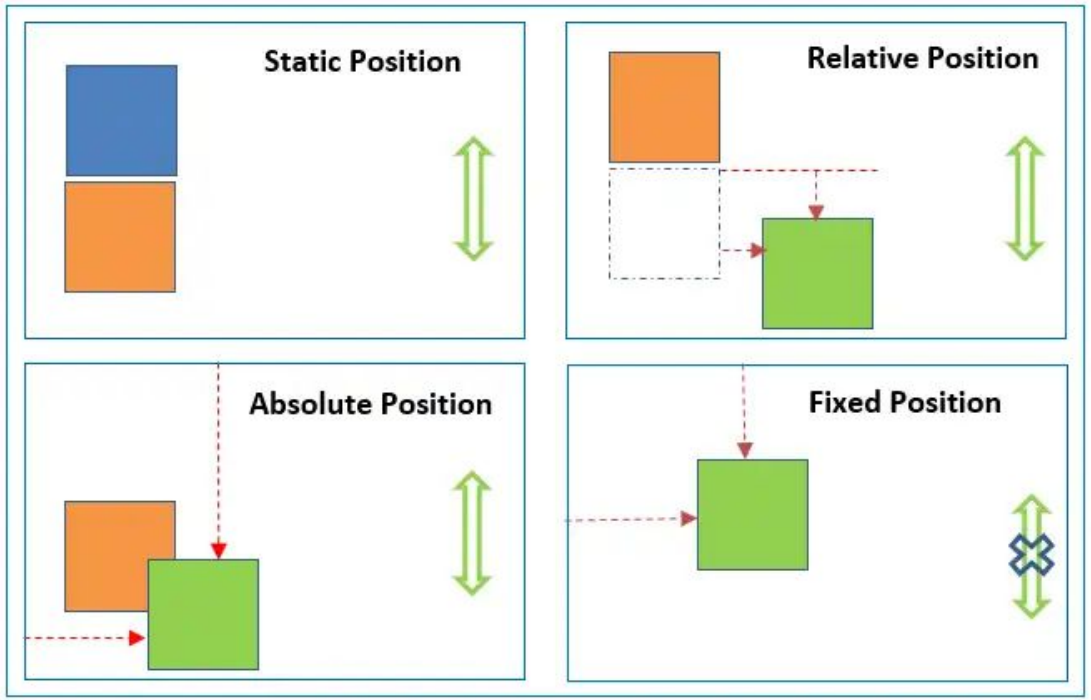
Incluía:

- cores
- fontes
- espaçamento
- alinhamento de texto



Fonte: Henryson, 2016

CSS: histórico



Fonte: Abrantes, 2023

CSS2 (1998)

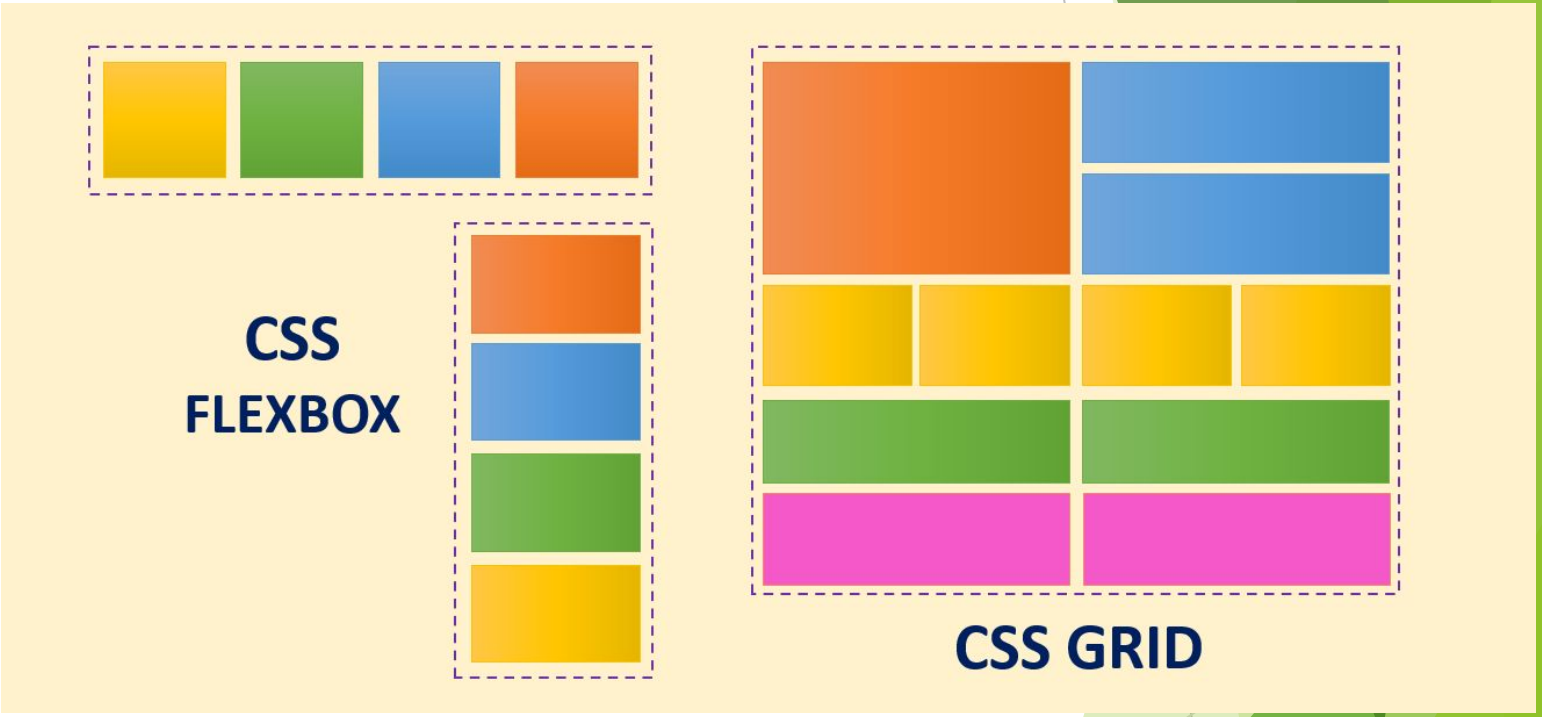
Adicionou recursos importantes como:

- posicionamento de elementos
- suporte para diferentes mídias
- melhor controle de layout

CSS: histórico

CSS3 (1999 - atual)

- flexbox
- grid layout
- animações
- transições
- sombras
- bordas arredondadas
- design responsivo



Fonte: Henryson, 2016

Diferente das versões anteriores, o CSS3 foi dividido em módulos, que são desenvolvidos separadamente.

CSS: módulos independentes

O World Wide Web Consortium decidiu dividir o CSS em módulos independentes. Cada módulo trata um assunto específico.

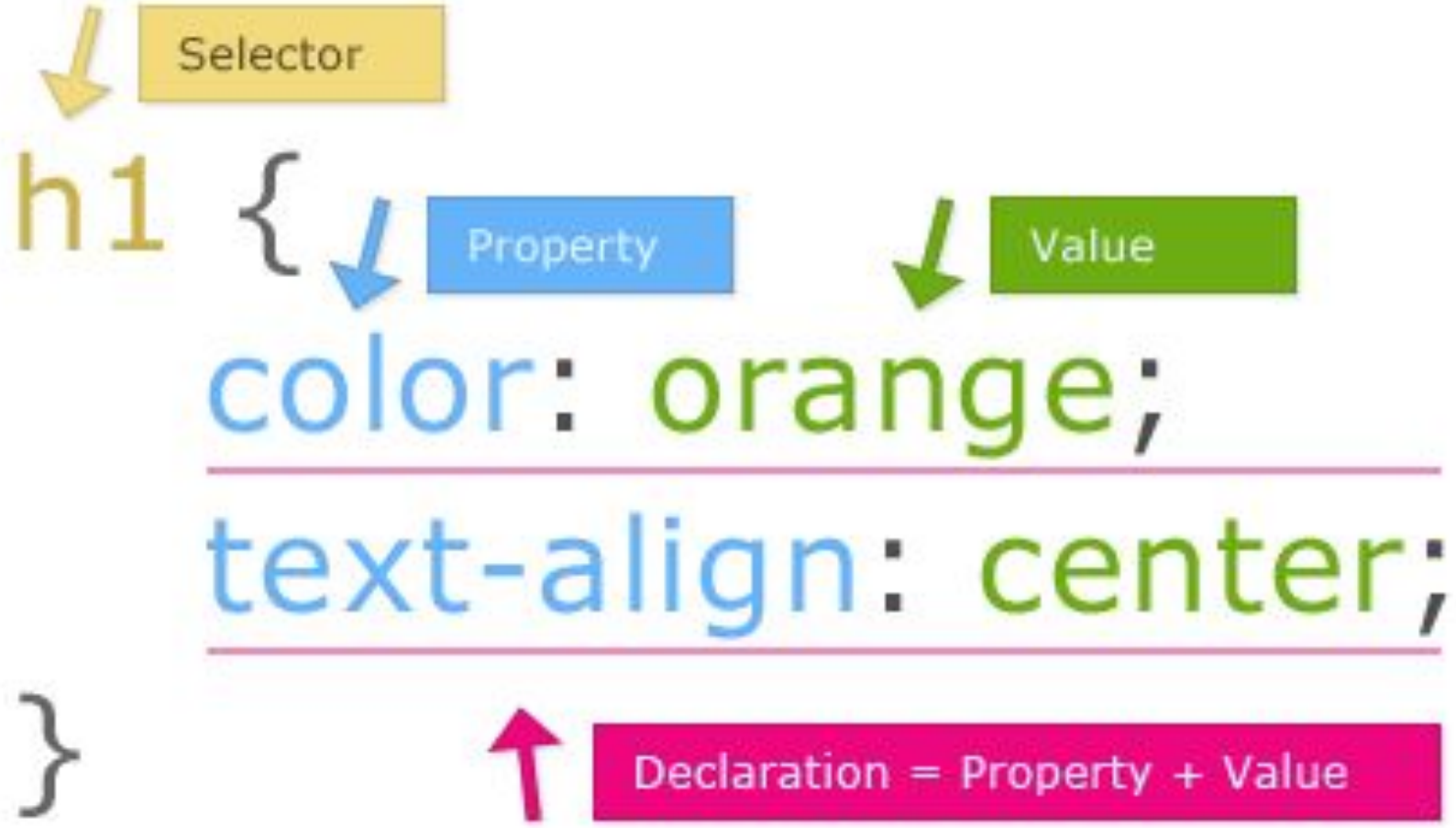
Exemplo:

- CSS Color
- CSS Flexbox
- CSS Grid
- CSS Animations
- CSS Transitions
- CSS Backgrounds and Borders
- CSS Fonts

Cada um desses módulos pode ser:

- criado
- atualizado
- padronizado
- independentemente dos outros.

Anatomia de uma regra css

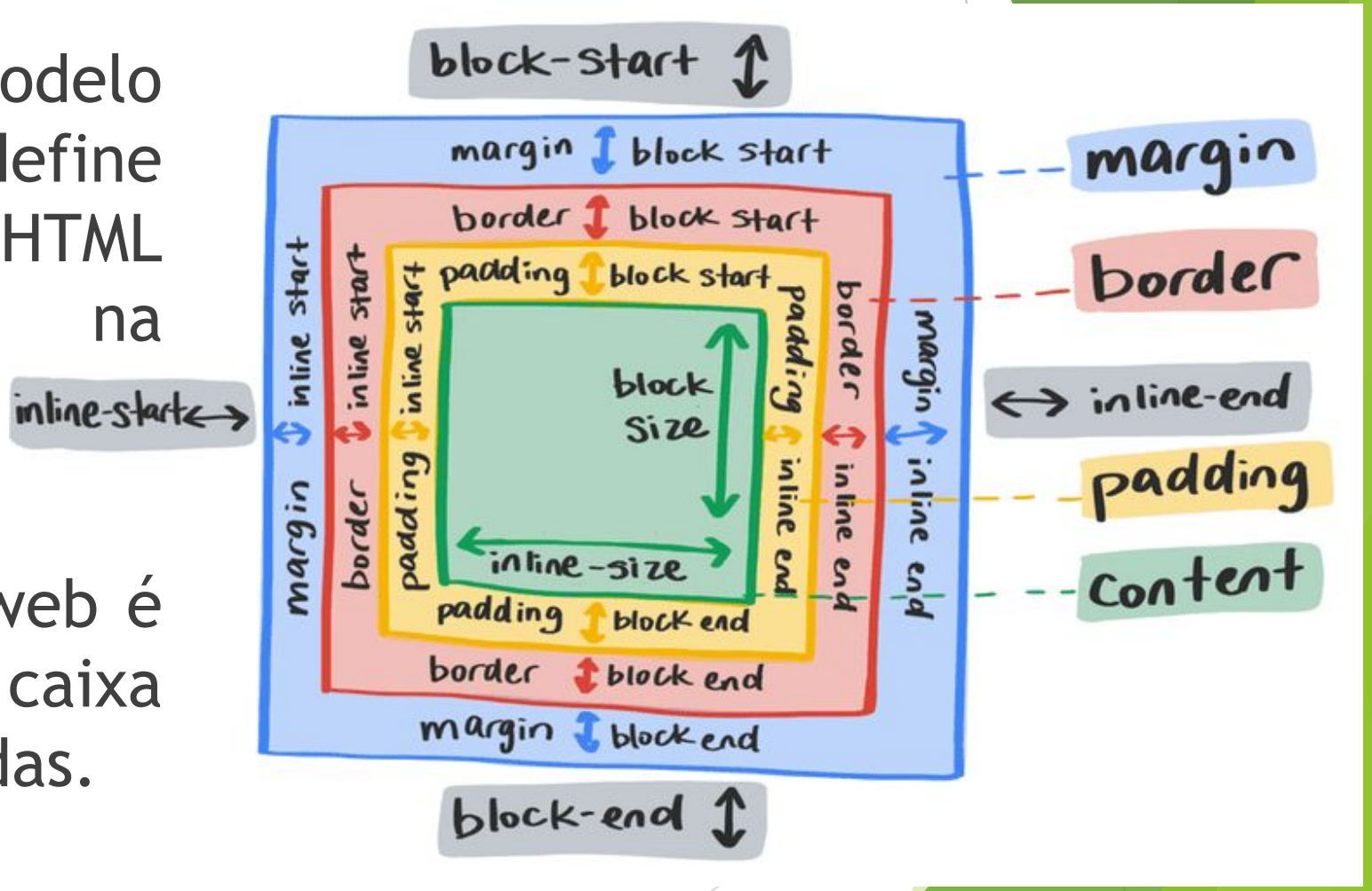


Fonte: Schiff, 2014

CSS Box Model

- O CSS Box Model (Modelo de Caixa do CSS), define como os elementos HTML ocupam espaço na página.

- Cada elemento na web é tratado como uma caixa composta por camadas.



Fonte: Schiff, 2014

CSS: página com css

Fonte: autora, 2026

CSS: página sem css

- Meu Espaço
-
- Editar perfil
- Configurações de interface
- Sair

- Comunidades

- Sobre a Univille
  - Institucional
  - Portal da Transparência
  - LGPD
  - Fale conosco
  - Trabalhe na Univille (Vagas também para PcD)
- Menu
  - Colégio
    - Colégio Joinville
    - Colégio São Bento do Sul
    - Colégio São Francisco do Sul
  - Cursos Técnicos
  - Graduação
    - Cursos
    - Cursos Graduação EaD
    - Contato Polos EaD
    - Renovação Uniedu
    - Estude na Univille
    - Bolsas de Estudo e Crédito Universitário
    - Mensalidades
    - Central de Atendimento Acadêmico
    - Central de Relacionamento com o Estudante
    - ENADE
    - PIBID

Fonte: autora, 2026

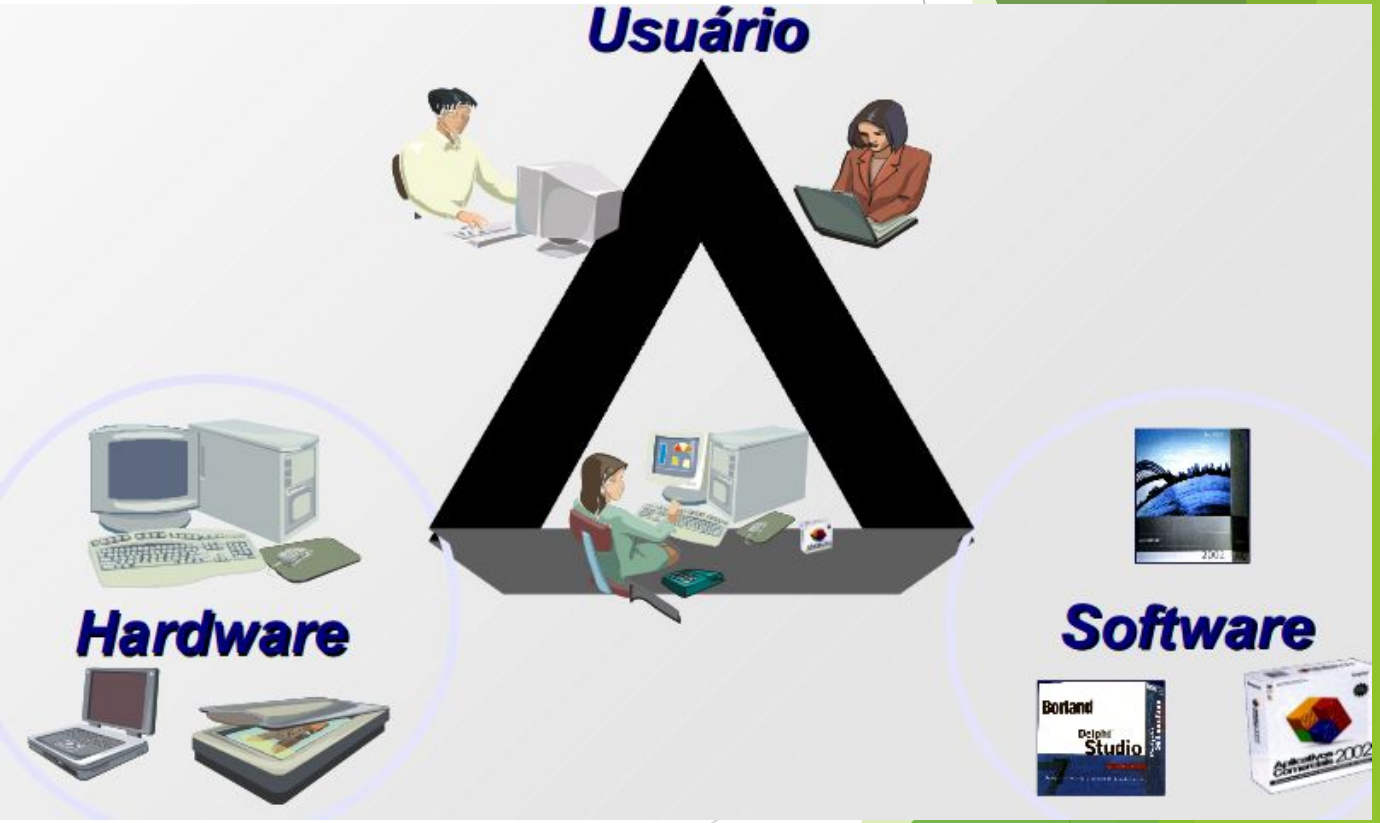
Desenvolvimento de aplicações web

Aula 5

Desenvolvimento de sistemas

- Criar um sistema computacional envolve **analisar** e **compreender** determinado problema, e **criar** uma solução para isso.

- Requer técnicas, métodos e mecanismos específicos para que elas funcionem conforme o esperado.



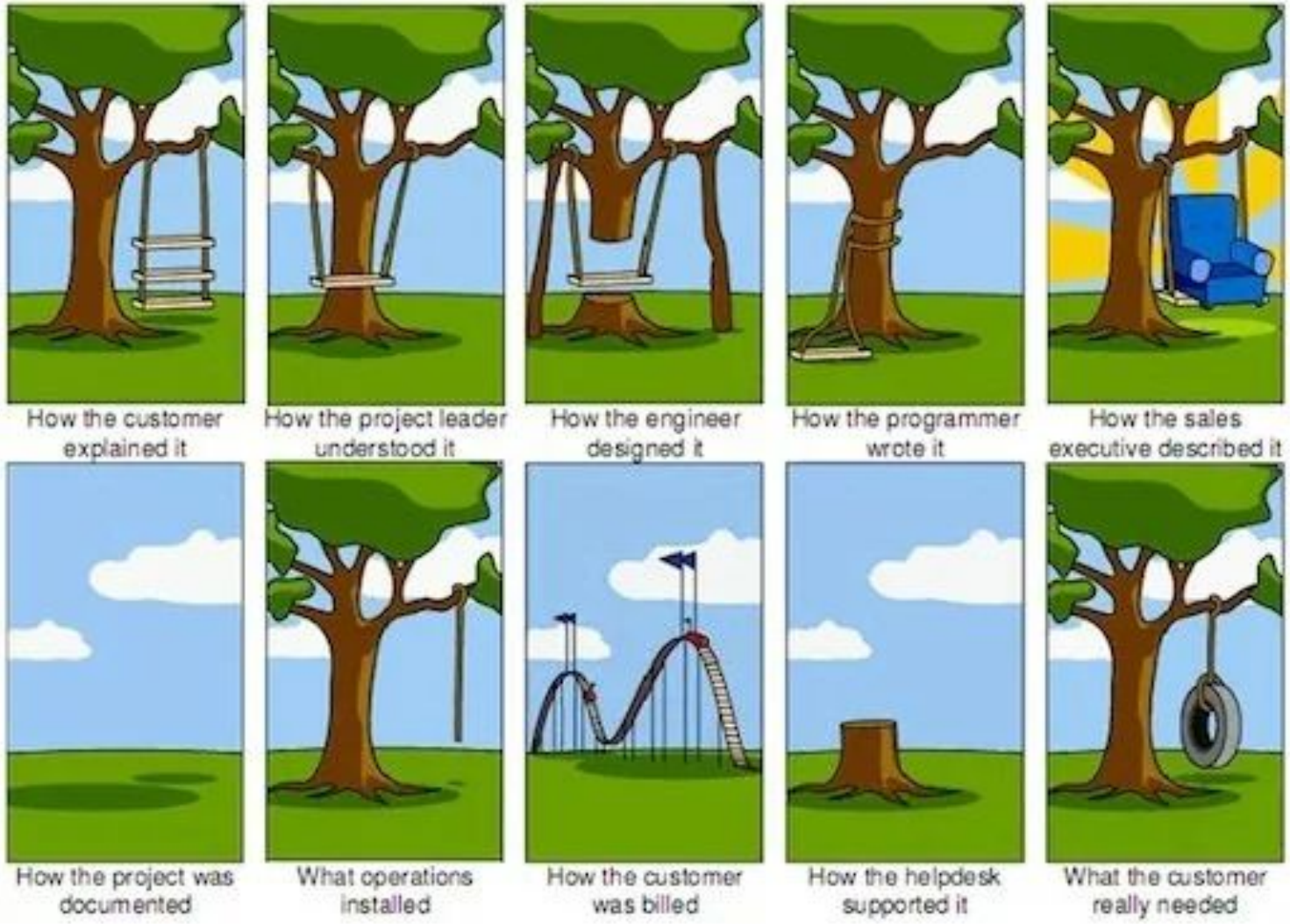
Fonte: Sapucaia, 2021

Desenvolvimento de sistemas

- Um dos maiores problemas que temos no dia a dia, é desenvolver soluções e funcionalidades que o usuário não quer, ou que não atendia totalmente a sua necessidade.

- O que causa quebra de expectativa e frustração no solicitante.

Desenvolvimento de sistemas



Aplicação web

- É um tipo de software que utiliza a internet como base de sua execução.

- Para projetar uma aplicação para uso na intranet ou internet, é necessário conhecer vários conceitos, que viemos estudando até o momento.

Aplicação web x site estático

- Uma aplicação web é diferente de um site estático, que não altera sua informação durante a execução da aplicação, e todos eu conteúdo está armazenado no arquivo html, entre as marcações das tags html.

- No **modelo estático**, as informações não vem de um servidor ou banco de dados (**dinâmico**), e sim, estão "**chumbadas**" no arquivo html.

Exemplo de tela com conteúdo estático

- Geralmente não é muito utilizado nos dias de hoje, e somente sites antigos ou com conteúdos imutáveis usam esse tipo de tela.

Estatísticas

8

Colunas	Soma	Média	Ocorrências	Maior	Menor
Código	47.423	267,93	177	584	1
COUNTE	31.329	177	1	177	177
VALOR	6.967.428,34	39.364	137	1.436.752,2	-2.500
PERCSIM	1	0,01	137	0,21	-0
QTD	193.802,55	1.094,93	82	100.203	0,08
RANK	15.753	89	177	177	1
PERC	8.900	50,28	177	100	0,56
VLRMED	722.399,45	4.081,35	132	400.000	-300

Fonte: Sankhya, 2024

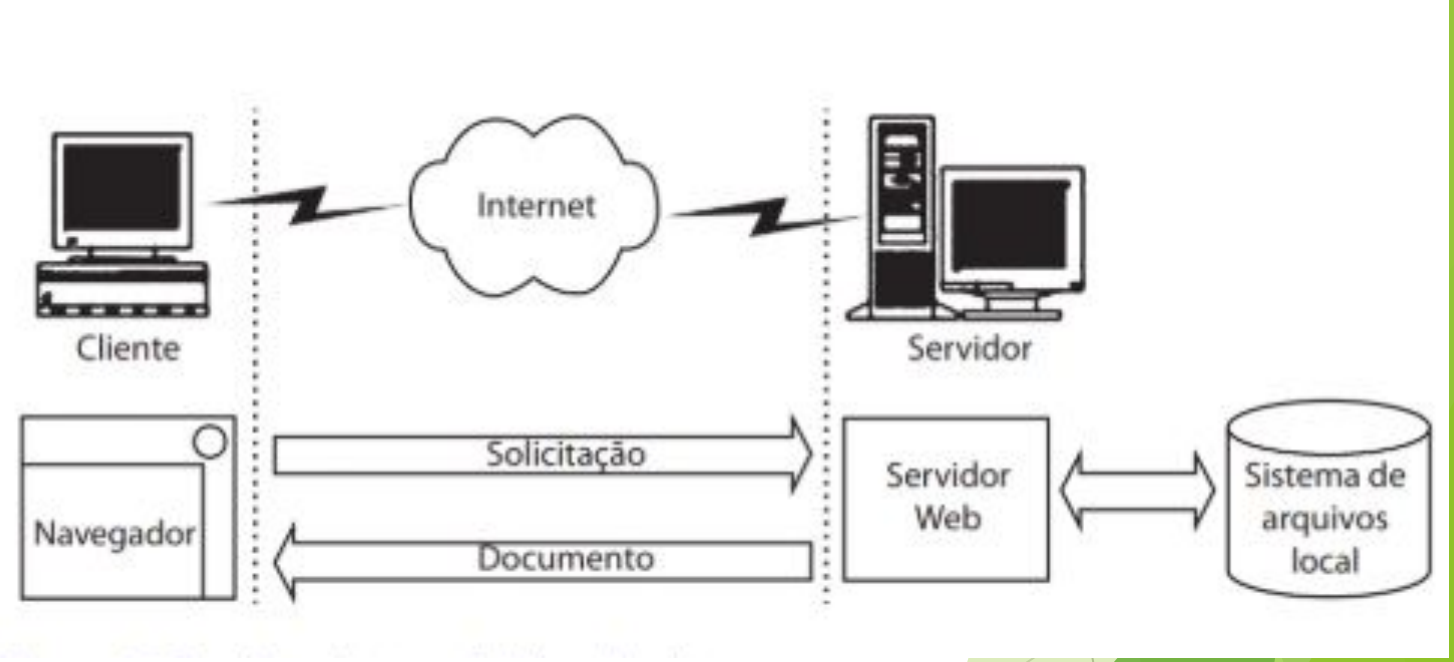
Aplicação web

Fonte: Sanroma, 2023

- Constrói seu conteúdo dinamicamente, com dados provenientes do banco de dados, processados a partir da interação do usuários com a interface.

Recapitulando sobre arquitetura web

- O servidor recebe a solicitação do cliente pelo navegador, e busca o documento requisitado em um sistema de arquivos e envia novamente ao navegador para ser exibido pelo cliente.



Fonte: MILETTO e BERTAGNOLLI, 2014

Aplicação web: processo de desenvolvimento

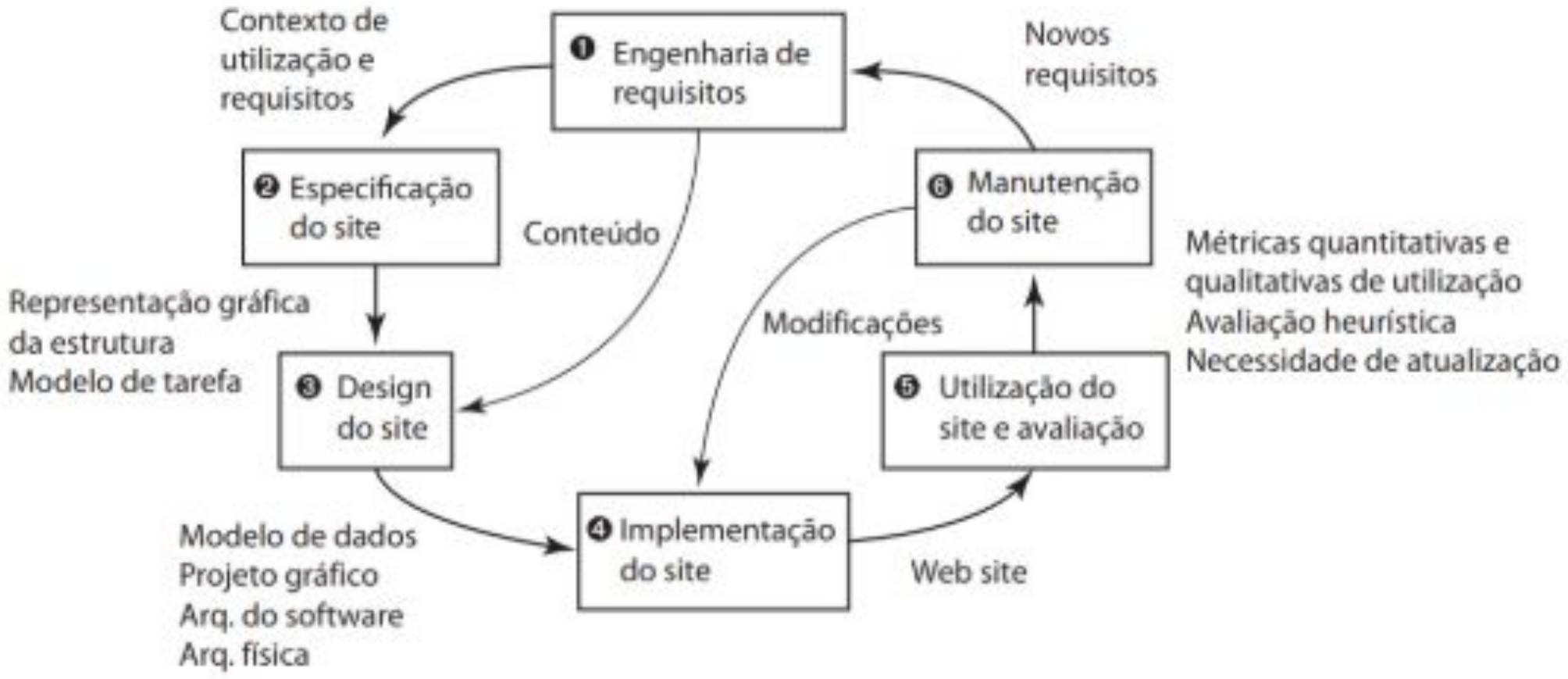
- O processo de desenvolvimento de software é um conjunto de atividades para compreender, modelar, desenvolver, testar, implantar e manter um produto de software.

- E existem diversos processos pelo qual uma aplicação de software web pode ser desenvolvida.

Aplicação web: processo de desenvolvimento

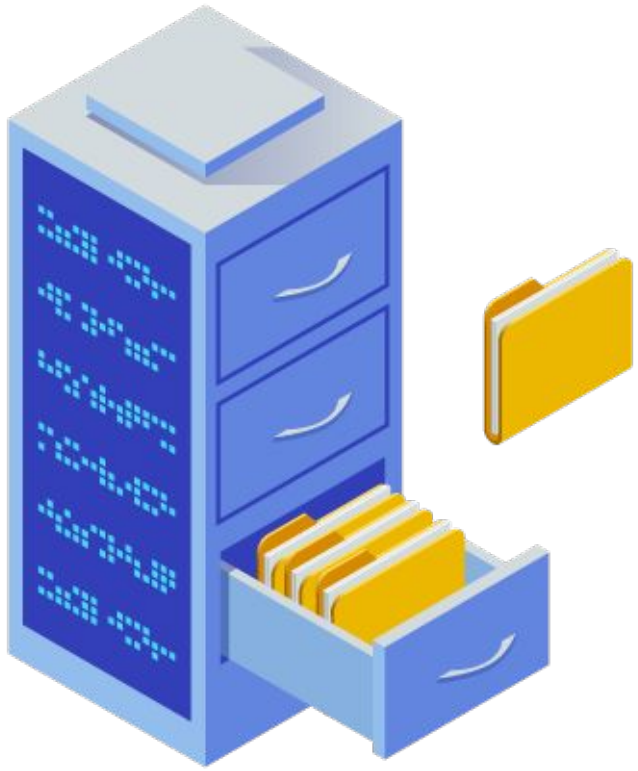
- Um dos principais pontos da execução da análise de um sistema é descobrir o que precisa ser feito.
- Primeiramente, precisa entender o problema para depois o quebrar em partes executáveis.
- E definir cada funcionalidade que precisa ser desenvolvida.

Aplicação web: processo de desenvolvimento



Fonte: MILETTO e BERTAGNOLLI, 2014

Aplicação web: armazenamento de dados

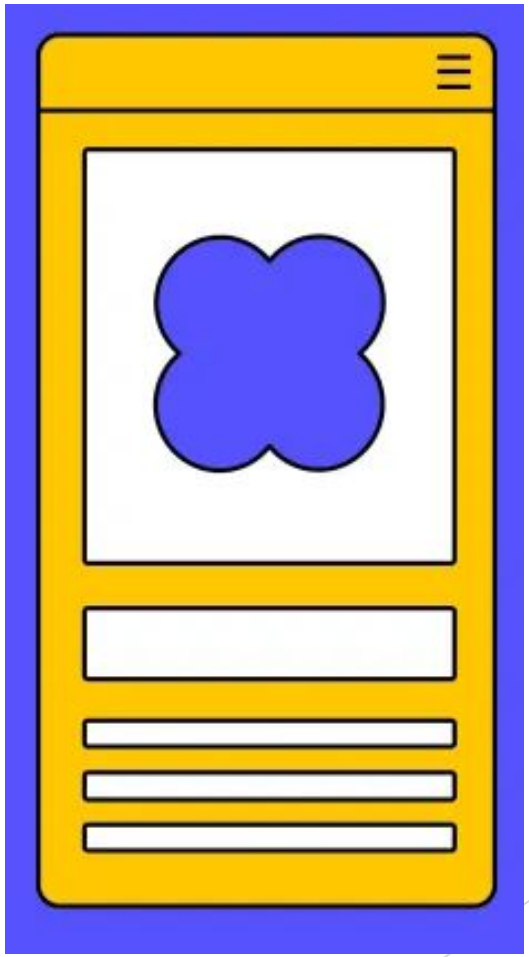


Fonte: univesp

- O armazenamento de dados de uma aplicação é realizada por um banco de dados que permite armazenar informações de forma permanente e sua manipulação.

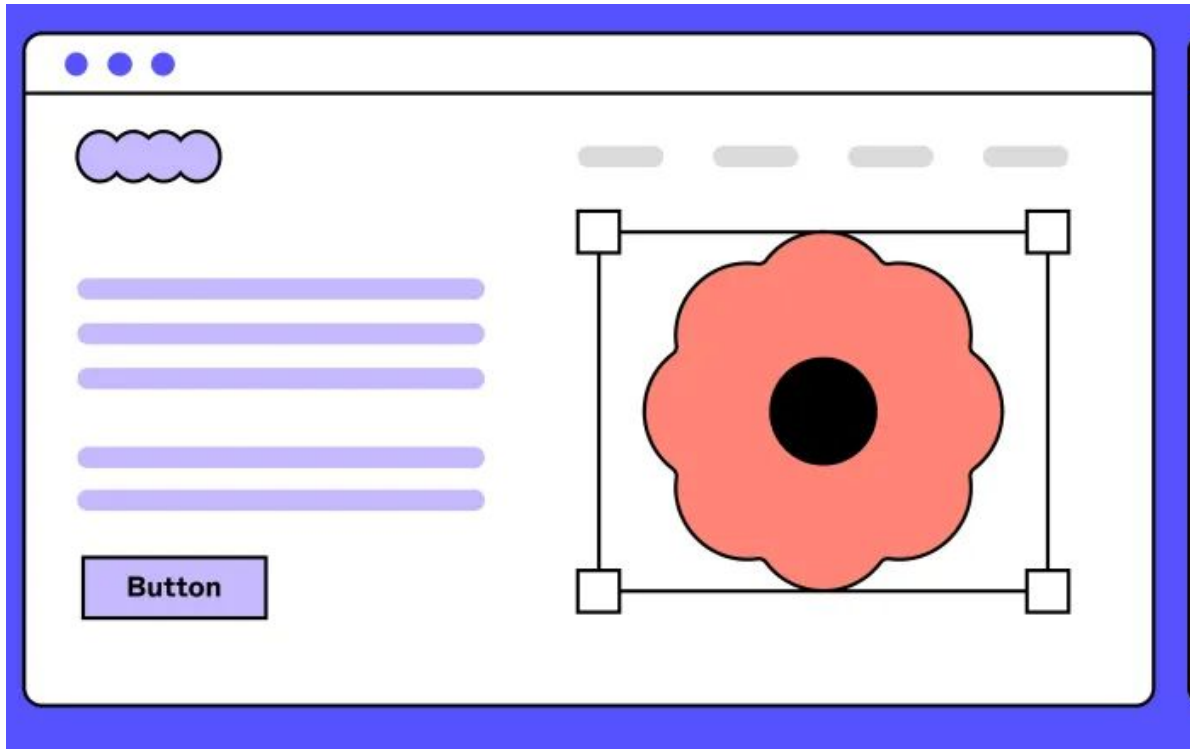
Aplicação web: tipos de interfaces

- Informativas:
- Quando mostram informações e avisos.



Fonte: Lazarini, 2024

Aplicação web: tipos de interfaces



Fonte: Lazarini, 2024

- Utilitárias:
- Quando permitem interação e captam informações do usuário.

Aplicação web: tipos de interfaces

- Uma boa interface é estável, eficaz, fácil de usar e eficiente, onde quem utiliza sente que ela foi desenvolvida centrada no usuário.

- Usuários ficam satisfeitos em usar uma interface que consuma pouco do seu esforço, tempo, seguindo a lógica de uso de seus usuários e que não apresente erros frequentes.

Aplicação web: concepção de interfaces

Engenharia de requisitos	A estrutura do site e o contexto de utilização são identificados. (Ver requisitos e descrições abordados no Capítulo 2.)
Especificação	Modelos da interface são construídos a partir dos requisitos obtidos durante a fase de análise.
Design	Modelos são construídos para que se possa refletir sobre o site e suas propriedades sem ter que implementá-lo.
Implementação	Criação de páginas HTML e de objetos de som/imagem necessários à aplicação. (Exemplos da implementação podem ser vistos no Capítulo 4.)
Utilização e avaliação	Usabilidade da interface e a sua coerência em relação aos requisitos iniciais são avaliadas.
Manutenção	Envolve um ciclo de maior duração, que abrange a coleta de novos requisitos e o planejamento das modificações identificadas durante a etapa de avaliação.

Fonte: MILETTO e BERTAGNOLLI, 2014

Aplicação web: prototipação

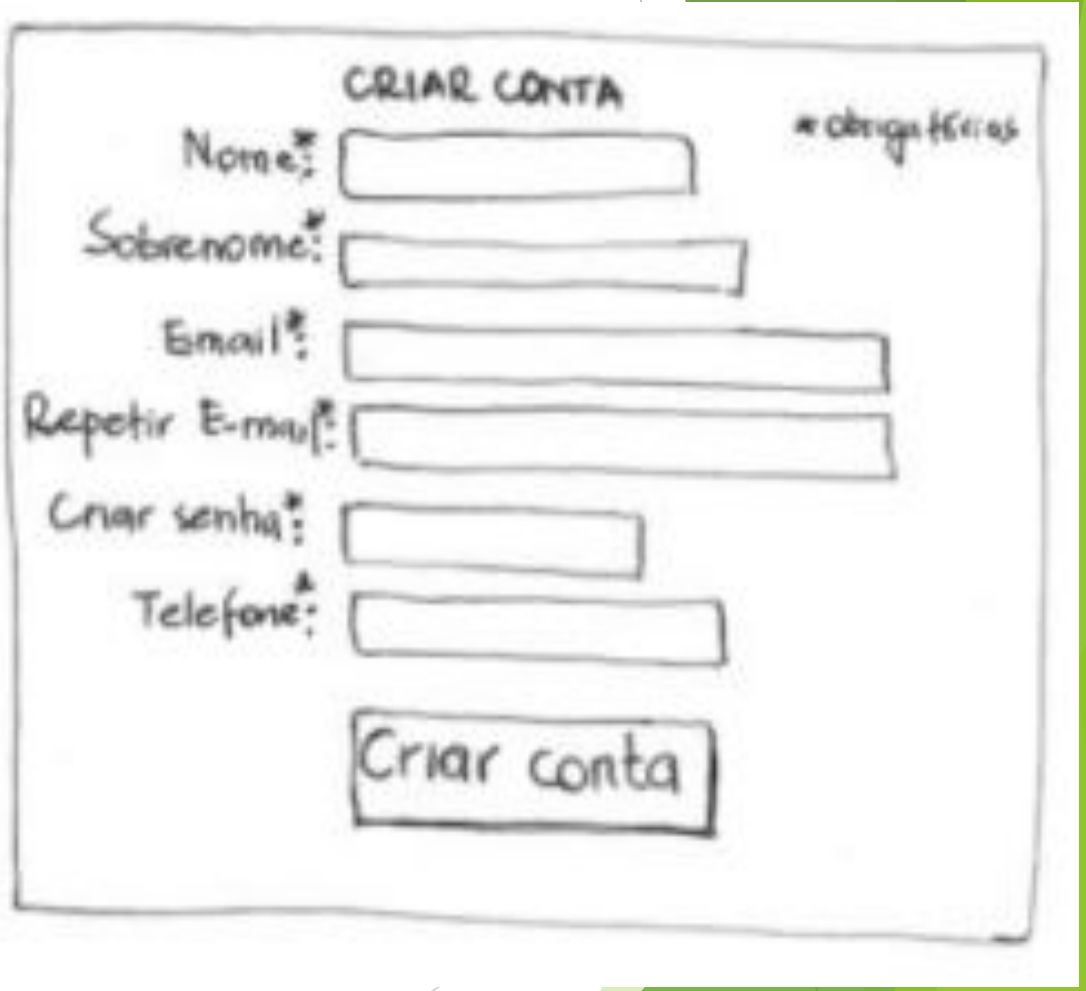
- A criação de protótipos das telas dos sistema permite entender melhor as funcionalidades pela visão do usuário e detalhar melhor o escopo das funcionalidades.

- Iniciamos com rascunhos iniciais de baixa fidelidade e vamos evoluindo para protótipos de alta fidelidade ao modelo final.

Aplicação web: prototipação

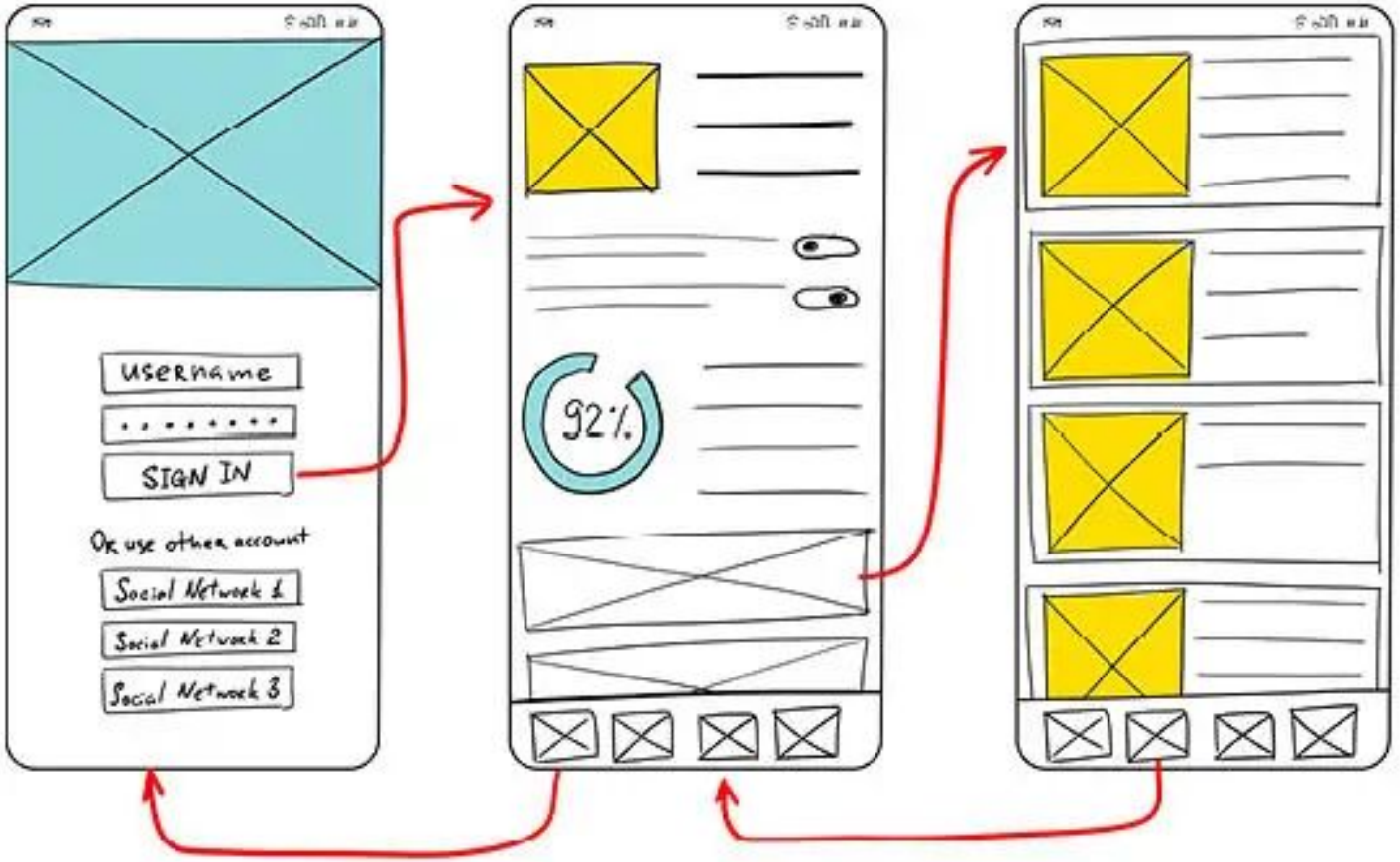
- Desta forma, o protótipo de baixa fidelidade é um **exemplo inicial** e **não finalizado** que serve para elucidar a comunicação entre as diferentes telas do sistema.

- Pode servir para auxiliar na detecção de lacunas, problemas ou ações possíveis não previstas nas telas.



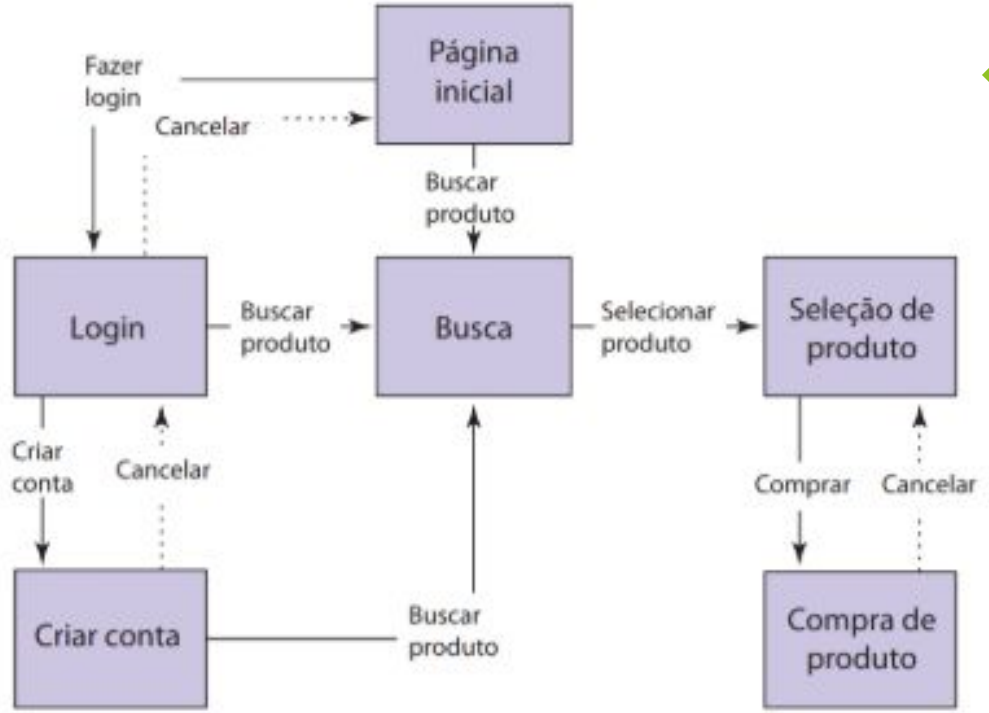
Fonte: MILETTO e BERTAGNOLLI, 2014

Mais exemplos de prototipação



Fonte: Lazarini, 2024

Aplicação web: navegação



Fonte: MILETTO e BERTAGNOLLI, 2014

- Neste outro diagrama colocamos nas caixas as páginas do sistema, e nas flechas as ações que vão ser executadas em cada uma, e os resultados de navegação.

Dúvidas?

Referência da aula

- ANDRÉ, João. Primeiros passos no desenvolvimento: front-end vs back-end. Medium, 13 dez. 2024. Disponível em: https://medium.com/@joaoandrej/primeiros-passos-no-desenvolvimento-front-end-vs-back-end-406c258a1f4d. Acesso em: 6 mar. 2026.

- GOTAPI. API endpoint example. GotAPI, s.d. Disponível em: https://gotapi.com/api-endpoint-example/. Acesso em: 6 mar. 2026.

- QODEX. What is an API endpoint. Qodex Blog, s.d. Disponível em: https://qodex.ai/blog/what-is-an-api-endpoint. Acesso em: 6 mar. 2026.

Referência da aula

- LEE, Ivan. What is an API endpoint? Examples & protection. Wallarm, 2025. Disponível em: https://www.wallarm.com/what/api-endpoint. Acesso em: 6 mar. 2026.

- ILYIN, Stepan. API endpoint monitoring: a comprehensive guide. Wallarm, 2025. Disponível em: https://www.wallarm.com/what/api-endpoint-monitoring. Acesso em: 6 mar. 2026.

- PARASHCHUK, Andriy. How to use background-size in CSS. Medium (JTWay), 17 jun. 2022. Disponível em: https://medium.com/jetthoughts/how-to-use-background-size-in-css-7b3dcdc941d5. Acesso em: 6 mar. 2026.

Referência da aula

- ADELEKE, David. CSS cascade layers. OpenReplay Blog, 2023. Disponível em: https://blog.openreplay.com/css-cascade-layers/. Acesso em: 6 mar. 2026.

- SCHIFF, Brad. What is CSS? LearnWebCode, s.d. Disponível em: https://learnwebcode.com/what-is-css/. Acesso em: 6 mar. 2026.

- GOOGLE DEVELOPERS. CSS layout. web.dev, 2021. Disponível em: https://web.dev/learn/css/layout?hl=pt. Acesso em: 6 mar. 2026.

- CRAZYLEAF DESIGN. Web design history: from cyberpunks to UI/UX.CrazyLeaf Design Blog, s.d. Disponível em: https://www.crazyleafdesign.com/blog/web-design-history-from-cyberpunks-to-uiux/. Acesso em: 6 mar. 2026.

Referência da aula

- ABRANTES, Felipe. CSS: Positions. DIO – Digital Innovation One, 3 mar. 2023. Disponível em: https://www.dio.me/articles/css-positions. Acesso em: 6 mar. 2026.

- LAZARINI, Douglas. O que é interface gráfica? Guia completo para iniciantes. Douglas Lazarini Blog, 25 jun. 2024. Disponível em: https://douglazarini.com/o-que-e-interface-grafica-guia-completo-para-iniciantes/. Acesso em: 6 mar. 2026

univille

univille.br